

WAD - Web Application Document

Projeto: Totem de Autoatendimento Socioassistencial - Prefeitura de Guarulhos

Repositorio: HackthonGuarulhos

Contexto: Hackathon Experimenta 2026 - Desafio Prefeitura de Guarulhos

Area: Servicos socioassistenciais, atendimento publico e inclusao digital

Aplicacao: API backend para totem de autoatendimento, consulta de beneficios, elegibilidade e agendamento

Sumario

1. [Introducao](#c1)
2. [Visao Geral da Aplicacao Web](#c2)
3. [Regras de Negocio](#c3)
4. [Requisitos do Sistema](#c4)
5. [Projeto Tecnico da Aplicacao Web](#c5)
6. [Banco de Dados](#c6)
7. [Rotas da API](#c7)
8. [Fluxos Principais](#c8)
9. [Como Executar o Projeto](#c9)
10. [Validacao e Testes](#c10)
11. [Matriz RF x RN x Endpoint](#c11)
12. [Matriz de Riscos](#c12)
13. [Conclusao e Trabalhos Futuros](#c13)
14. [Referencias](#c14)

1. Introducao

O projeto **Totem de Autoatendimento Socioassistencial** foi desenvolvido como uma solucao para apoiar a Prefeitura de Guarulhos na organizacao, padronizacao e ampliao do acesso aos servicos socioassistenciais.

O desafio identificado esta relacionado a dificuldade de pessoas em situacao de vulnerabilidade acessarem informacoes sobre beneficios, entenderem se possuem elegibilidade e realizarem agendamentos de atendimento de maneira simples. Em muitos casos, o atendimento depende de sistemas fragmentados, deslocamento ate unidades publicas e orientacoes presenciais que poderiam ser parcialmente automatizadas.

A aplicacao proposta funciona como uma API backend para um totem ou interface de autoatendimento. Ela permite:

- ? iniciar um atendimento;
- ? consultar beneficios disponiveis;
- ? registrar informacoes familiares;
- ? calcular beneficios potencialmente elegiveis;
- ? selecionar beneficios de interesse;
- ? consultar datas e horarios disponiveis;
- ? registrar agendamentos;
- ? exibir documentos necessarios para cadastro conforme beneficios selecionados;
- ? acompanhar dados de atendimento.

O sistema utiliza banco de dados PostgreSQL hospedado no Supabase, com conexão configurada por variáveis de ambiente e migrations para criação das tabelas necessárias.

2. Visao Geral da Aplicacao Web

2.1 Problema

A Secretaria de Desenvolvimento Social precisa lidar com um volume alto de atendimentos e com uma população que, muitas vezes, não sabe quais programas sociais existem, quais critérios precisa atender ou como agendar atendimento.

Os principais problemas observados são:

- ? dificuldade de acesso a informação sobre beneficios;
- ? falta de padronização no inicio do atendimento;
- ? dependência excessiva de atendimento humano para duvidas basicas;
- ? baixa visibilidade sobre elegibilidade;
- ? fragmentacao entre orientacao, triagem e agendamento;
- ? risco de perda de dados quando o processo e feito manualmente.

2.2 Objetivo do Projeto

O objetivo do projeto e oferecer uma camada backend para um sistema de autoatendimento capaz de orientar cidadaos, registrar informacoes basicas, sugerir beneficios compatíveis e permitir agendamento em horarios disponiveis.

A solução não substitui o atendimento humano, mas organiza a primeira etapa da jornada, reduzindo fricções e preparando melhor o atendimento presencial ou remoto.

2.3 Escopo

O escopo atual contempla:

- ? API REST em Node.js com Express;

- ? conexao com banco PostgreSQL/Supabase;
- ? criacao automatica das tabelas por migration;
- ? cadastro inicial de faixas de renda;
- ? cadastro inicial de beneficios;
- ? cadastro inicial de slots de agendamento;
- ? regras de elegibilidade baseadas em renda e composicao familiar;
- ? registro de atendimento e dados familiares;
- ? selecao de beneficios;
- ? agendamento de atendimento;
- ? frontend simples em HTML, CSS e JavaScript para testar o fluxo completo;
- ? exibicao de documentos necessarios para cadastro com base no PDF de programas sociais;
- ? verificacao de saude da API e conexao com banco.

Fora do escopo atual:

- ? autenticação de usuários administrativos;
- ? painel completo de gestao operacional;
- ? integração oficial com sistemas internos da prefeitura;
- ? envio real de SMS, WhatsApp ou e-mail;
- ? validação documental automatica;
- ? analise juridica final dos criterios de cada beneficio.

2.4 Publico-Alvo

O sistema foi pensado para:

- ? cidadaos de Guarulhos em busca de beneficios socioassistenciais;
- ? familias em situacao de vulnerabilidade social;
- ? atendentes e equipes da Secretaria de Desenvolvimento Social;
- ? gestores publicos que precisam organizar fluxos de atendimento;
- ? unidades de atendimento que precisam reduzir filas e padronizar triagens.

2.5 Solucao

A solucao e uma API backend que centraliza os dados principais do fluxo de autoatendimento. A aplicacao recebe dados do usuario, consulta regras cadastradas no banco, calcula elegibilidade e cria registros de atendimento e agendamento.

O backend segue uma divisao em camadas:

- ? **Routes:** definem os endpoints HTTP;
- ? **Controllers:** recebem as requisicoes e retornam respostas;
- ? **Services:** concentram regras de negocio;

- ? **Repositories:** acessam o banco de dados;
- ? **Database:** gerencia conexao PostgreSQL/Supabase;
- ? **Migrations:** criam e populam as tabelas iniciais.

2.6 Proposta de Valor

Elemento	Descrição
Público	Cidadãos que precisam acessar benefícios sociais e atendimento público
Dor	Falta de clareza sobre benefícios, critérios e agendamento
Solução	Totem/API de autoatendimento com triagem, elegibilidade e agendamento
Ganho	Atendimento mais organizado, rápido e padronizado
Diferencial	Fluxo único conectando informação, regra de negócio e persistência no banco

3. Regras de Negócio

As regras abaixo foram definidas de acordo com o funcionamento atual do sistema e com o contexto socioassistencial do projeto.

RN01 - Criação de atendimento

Todo atendimento iniciado deve ser registrado no banco com status inicial **em_andamento**.

Quando o atendimento é criado, o sistema também define valores iniciais padrão para composição familiar:

- ? quantidade de pessoas: 1;
- ? quantidade de adultos: 1;
- ? quantidade de crianças: 0;
- ? quantidade de idosos: 0;
- ? possui pessoa com deficiência: falso.

RN02 - Dados familiares obrigatórios para análise

Para atualizar os dados familiares de um atendimento, o sistema precisa receber informações mínimas que permitam avaliar o perfil socioeconômico.

Esses dados incluem:

- ? faixa de renda;
- ? quantidade de pessoas;
- ? quantidade de adultos;
- ? quantidade de crianças;

- ? quantidade de idosos;
- ? indicacao sobre pessoa com deficiencia;
- ? observacoes complementares, quando houver.

RN03 - Faixa de renda deve existir e estar ativa

A faixa de renda utilizada em um atendimento deve existir no banco e estar ativa. Isso evita que o sistema calcule elegibilidade com base em categorias inexistentes ou descontinuadas.

RN04 - Beneficios retornados devem estar ativos

Ao listar beneficios para o usuario, o sistema deve retornar apenas beneficios ativos. Beneficios inativos nao devem aparecer para escolha ou recomendacao.

RN05 - Elegibilidade deve considerar renda e perfil familiar

A recomendacao de beneficios deve considerar:

- ? faixa de renda da familia;
- ? quantidade de pessoas;
- ? presenca de criancas;
- ? presenca de idosos;
- ? presenca de pessoa com deficiencia;
- ? regras especificas cadastradas para cada beneficio.

RN06 - Payload de beneficios selecionados deve ser uma lista

Ao selecionar beneficios de interesse, o sistema deve receber uma lista de identificadores de beneficios. Caso o formato seja invalido, a requisicao deve ser recusada.

RN07 - Agendamento exige dados de contato

Para criar um agendamento, o sistema deve receber:

- ? nome do solicitante;
- ? telefone;
- ? data;
- ? horario.

Essas informacoes sao necessarias para identificar o cidadao e permitir contato posterior.

RN08 - Agendamento depende de slot disponivel

Um agendamento so pode ser criado quando existir slot ativo para a data e horario informados.

O slot também precisa possuir vagas disponíveis. Caso a capacidade máxima tenha sido atingida, o sistema não deve permitir novo agendamento para aquele horário.

RN09 - Ao agendar, o atendimento deve ser concluído

Quando um atendimento recebe um agendamento válido, seu status deve ser atualizado para **concluído**.

Essa regra indica que o fluxo de autoatendimento terminou e que o usuário já possui encaminhamento para atendimento.

RN10 - Controle de vagas deve ser persistido no banco

Ao criar um agendamento, o sistema deve incrementar a quantidade de vagas ocupadas do slot correspondente.

Essa regra impede que múltiplos usuários sejam agendados acima da capacidade permitida.

RN11 - Erros internos não devem expor informações sensíveis

Falhas internas, principalmente relacionadas ao banco de dados, não devem retornar detalhes técnicos sensíveis para o usuário final.

O sistema deve retornar mensagens controladas e registrar o erro no backend para depuração.

RN12 - Conexão com banco depende de variável de ambiente

A aplicação só deve iniciar corretamente quando a variável de ambiente `DATABASE_URL` estiver configurada.

Essa regra garante que o backend utilize um banco real e não dependa de dados locais inconsistentes.

RN13 - Documentos devem ser exibidos após seleção de benefícios ou agendamento

Depois que o usuário seleciona benefícios de interesse e/ou conclui o agendamento, o frontend deve apresentar uma lista de documentos necessários para cadastro.

A lista deve considerar o benefício selecionado e utilizar como referência o PDF `documentos_programas_sociais.pdf`. Quando o benefício não possuir correspondência específica no PDF, o sistema deve exibir uma lista geral de documentos e orientar o usuário a confirmar as exigências no CRAS ou órgão responsável.

Programas mapeados atualmente:

- ? Bolsa Família / Cadastro Único;
- ? Tarifa Social de Energia Elétrica;
- ? Seguro-Desemprego;
- ? BPC / LOAS;
- ? Carteira da Pessoa Idosa / Passe Livre;

- ? Auxilio Gas;
- ? Minha Casa Minha Vida, caso seja cadastrado como beneficio futuro;
- ? documentos gerais para beneficios sem correspondencia direta.

4. Requisitos do Sistema

4.1 Requisitos Funcionais

ID	Requisito Funcional	Descricao
RF01	Iniciar atendimento	O sistema deve permitir criar um novo atendimento.
RF02	Consultar beneficios	O sistema deve listar beneficios ativos disponiveis.
RF03	Consultar faixas de renda	O sistema deve listar faixas de renda ativas.
RF04	Atualizar dados familiares	O sistema deve salvar dados familiares de um atendimento.
RF05	Calcular beneficios elegiveis	O sistema deve indicar beneficios compatíveis com o perfil informado.
RF06	Selecionar beneficios	O sistema deve permitir associar beneficios de interesse ao atendimento.
RF07	Consultar beneficios de atendimento	O sistema deve listar beneficios associados a um atendimento.
RF08	Consultar datas disponiveis	O sistema deve listar datas com slots de agendamento.
RF09	Consultar slots disponiveis	O sistema deve listar horarios disponiveis por data.
RF10	Criar agendamento	O sistema deve permitir agendar atendimento em slot disponivel.
RF11	Consultar atendimento	O sistema deve retornar dados de um atendimento existente.
RF12	Verificar saude da API	O sistema deve possuir endpoint de health check com validacao do banco.
RF13	Exibir documentos necessarios	O frontend deve mostrar os documentos de cadastro conforme os beneficios selecionados.

4.2 Requisitos Nao Funcionais

ID	Requisito Nao Funcional	Descricao
RNF01	Persistencia real	O sistema deve utilizar banco PostgreSQL/Supabase real.
RNF02	Configuracao segura	Credenciais devem ficar em .env, sem serem expostas no codigo.
RNF03	API REST	A comunicacao deve ocorrer por endpoints HTTP em JSON.
RNF04	Separacao em camadas	O projeto deve separar rotas, controllers, services e repositories.

ID	Requisito Nao Funcional	Descricao
RNF05	Migracoes idempotentes	O banco deve ser preparado por scripts que podem ser executados mais de uma vez.
RNF06	Portabilidade	A aplicacao deve executar localmente com Node.js e variaveis de ambiente.
RNF07	Tratamento de erros	Erros devem ser tratados com respostas adequadas ao cliente.
RNF08	Manutenibilidade	A estrutura deve facilitar futuras regras, endpoints e integracoes.
RNF09	Interface simples de teste	O sistema deve disponibilizar uma tela estatica servida pelo Express para validar o fluxo completo.

5. Projeto Tecnico da Aplicacao Web

5.1 Arquitetura

O backend foi organizado em camadas para separar responsabilidades e facilitar manutencao.

```

flowchart TD
    A["Cliente / Totem"] --> P["Frontend estatico HTML/CSS/JS"]
    P --> B["Rotas Express"]
    B --> C["Controllers"]
    C --> D["Services"]
    D --> E["Repositories"]
    E --> F["Banco PostgreSQL / Supabase"]
    G["Migrations"] --> F
    P --> H["Lista de documentos por beneficio"]

```

5.2 Tecnologias Utilizadas

Tecnologia	Uso no Projeto
Node.js	Ambiente de execucao backend
Express	Criacao da API HTTP
HTML	Estrutura da interface simples de teste
CSS	Estilizacao responsiva do frontend
JavaScript	Consumo da API, controle do fluxo e exibicao de documentos
PostgreSQL	Banco de dados relacional
Supabase	Hospedagem do banco PostgreSQL
pg	Driver de conexao com PostgreSQL
dotenv	Carregamento de variaveis de ambiente
TypeScript compiler	Validacao estatica do JavaScript com allowJs

5.3 Estrutura de Pastas

```

src/
  ■■■ config/
  ■ ■■■ database.js

```



```

■■■ controllers/
■   ■■■ AtendimentoController.js
■   ■■■ BeneficioController.js
■■■ middlewares/
■   ■■■ errorHandler.js
■■■ migrations/
■   ■■■ index.js
■■■ repositories/
■   ■■■ AgendamentoRepository.js
■   ■■■ AtendimentoRepository.js
■   ■■■ BeneficioRepository.js
■■■ routes/
■   ■■■ index.js
■   ■■■ server.js
■■■ Services/
■   ■■■ AtendimentoService.js
■   ■■■ ElegibilidadeService.js
■■■ package.json
■■■ tsconfig.json

```

Observacao: a pasta src/public/ tambem faz parte da aplicacao e contem o frontend simples utilizado para testar o sistema:

- ? index.html: estrutura da tela;
- ? styles.css: estilos e responsividade;
- ? app.js: consumo da API, controle do fluxo e exibicao dos documentos.

5.4 Responsabilidades por Camada

Camada	Responsabilidade
routes	Define os caminhos HTTP disponiveis na API.
controllers	Recebe requisicoes, chama services e retorna respostas JSON.
Services	Aplica regras de negocio e orquestra operacoes.
repositories	Executa consultas e comandos no banco de dados.
config	Configura conexao com PostgreSQL/Supabase.
migrations	Cria tabelas e dados iniciais.
public	Contem a tela HTML/CSS/JS usada para testar atendimento, beneficios, agendamento e documentos.
middlewares	Centraliza tratamento de erros.

6. Banco de Dados

6.1 Visao Geral

O sistema utiliza PostgreSQL hospedado no Supabase. A conexao e feita pela variavel de ambiente DATABASE_URL, carregada com dotenv.

O banco guarda:

- ? faixas de renda;
- ? beneficios disponiveis;

- ? atendimentos;
- ? beneficios selecionados por atendimento;
- ? slots de agendamento;
- ? agendamentos realizados.

6.2 Modelo de Dados

O diagrama abaixo representa o relacionamento entre as principais entidades do sistema, incluindo atendimento, beneficios, agendamentos e documentos exigidos por beneficio.

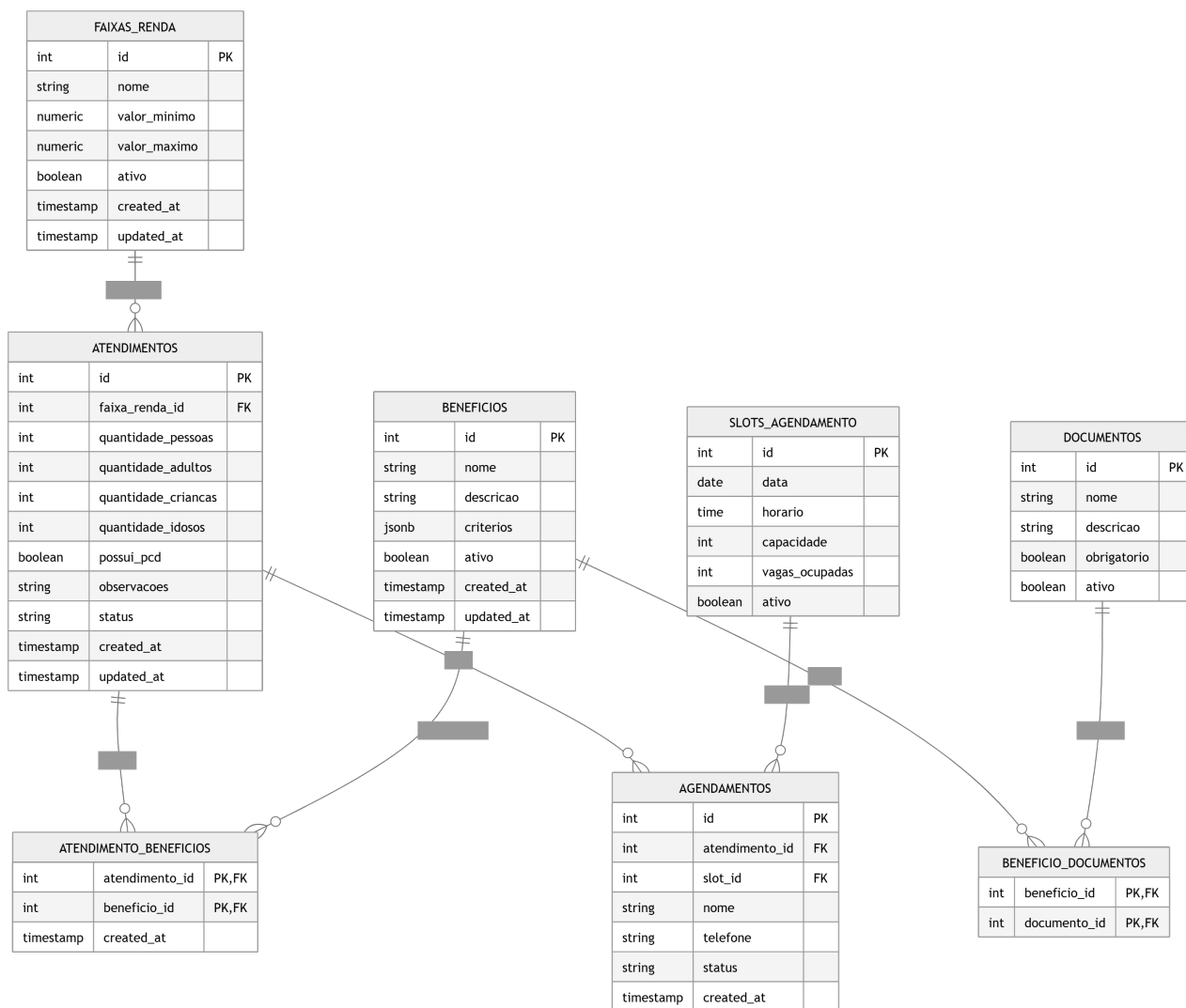


Diagrama de entidade relacionamento

6.3 Modelo ER em Mermaid

O modelo entidade-relacionamento do sistema organiza o fluxo de autoatendimento em torno da entidade **ATENDIMENTOS**. Essa entidade representa cada atendimento iniciado pelo usuário no totem e concentra as informações familiares usadas na triagem, como faixa de renda, quantidade de pessoas, composição familiar, condições declaradas e status do processo.

A entidade **FAIXAS_RENDA** classifica socioeconomicamente os atendimentos. O relacionamento entre **FAIXAS_RENDA** e **ATENDIMENTOS** indica que uma faixa de renda pode estar associada a varios atendimentos, enquanto cada atendimento pertence a uma unica faixa de renda. Essa relacao e importante para aplicar regras de elegibilidade e sugerir beneficios coerentes com o perfil informado.

A entidade **BENEFICIOS** armazena os programas sociais disponiveis no sistema, como Bolsa Familia, Tarifa Social, BPC/LOAS e outros beneficios cadastrados. Como um atendimento pode estar relacionado a varios beneficios e um beneficio pode aparecer em varios atendimentos, o modelo utiliza a tabela associativa **ATENDIMENTO_BENEFICIOS**. Essa tabela resolve o relacionamento muitos-para-muitos entre **ATENDIMENTOS** e **BENEFICIOS**, permitindo registrar quais beneficios foram considerados elegiveis ou selecionados em cada atendimento.

O agendamento e representado pelas entidades **SLOTS_AGENDAMENTO** e **AGENDAMENTOS**. A entidade **SLOTS_AGENDAMENTO** guarda as datas, horarios, capacidade e quantidade de vagas ocupadas. Ja a entidade **AGENDAMENTOS** registra a reserva feita pelo usuario, vinculando o atendimento a um horario disponivel. Assim, um atendimento pode gerar um agendamento, e cada agendamento utiliza um slot previamente cadastrado. Essa separacao permite controlar disponibilidade, evitar excesso de vagas e manter historico dos atendimentos agendados.

O diagrama tambem inclui as entidades **DOCUMENTOS** e **BENEFICIO_DOCUMENTOS**. **DOCUMENTOS** representa os documentos necessarios para cadastro, como RG, CPF, comprovante de residencia, Cadastro Unico, laudos medicos e comprovantes de renda. Como um beneficio pode exigir varios documentos e um mesmo documento pode ser exigido por varios beneficios, a tabela **BENEFICIO_DOCUMENTOS** resolve esse relacionamento muitos-para-muitos. Essa estrutura permite que a aplicacao mostre ao usuario, apos a selecao de beneficios e agendamento, quais documentos devem ser levados ao atendimento.

De forma geral, o modelo conecta triagem, elegibilidade, selecao de beneficios, agendamento e orientacao documental. As tabelas principais armazenam os dados centrais do processo, enquanto as tabelas associativas garantem flexibilidade para representar multiplos beneficios por atendimento e multiplos documentos por beneficio.

```
erDiagram
    FAIXAS_RENDA ||--o{ ATENDIMENTOS : classifica
    ATENDIMENTOS ||--o{ ATENDIMENTO_BENEFICIOS : possui
    BENEFICIOS ||--o{ ATENDIMENTO_BENEFICIOS : selecionado
    ATENDIMENTOS ||--o{ AGENDAMENTOS : gera
    SLOTS_AGENDAMENTO ||--o{ AGENDAMENTOS : reserva

    FAIXAS_RENDA {
        int id
        string nome
        numeric valor_minimo
        numeric valor_maximo
        boolean ativo
    }

    BENEFICIOS {
        int id
        string nome
        string descricao
        jsonb criterios
        boolean ativo
    }

    ATENDIMENTOS {
        int id
        int faixa_renda_id
        int quantidade_pessoas
        int quantidade_adultos
        int quantidade_crianças
        int quantidade_idosos
        boolean possui_pcd
        string status
    }

    ATENDIMENTO_BENEFICIOS {
        int atendimento_id
```

```

        int beneficio_id
    }

    SLOTS_AGENDAMENTO {
        int id
        date data
        time horario
        int capacidade
        int vagas_ocupadas
        boolean ativo
    }

    AGENDAMENTOS {
        int id
        int atendimento_id
        int slot_id
        string nome
        string telefone
        string status
    }
}

```

6.4 Tabelas

faixas_renda

Armazena categorias de renda utilizadas para classificacao socioeconomica.

Campos principais:

```

? id;
? nome;
? valor_minimo;
? valor_maximo;
? ativo;
? created_at;
? updated_at.

```

beneficios

Armazena beneficios socioassistenciais disponiveis no sistema.

Campos principais:

```

? id;
? nome;
? descricao;
? criterios;
? ativo;
? created_at;
? updated_at.

```

atendimentos

Armazena cada fluxo iniciado pelo usuario.

Campos principais:

? id;
? faixa_renda_id;
? quantidade_pessoas;
? quantidade_adultos;
? quantidade_crianças;
? quantidade_idosos;
? possui_pcd;
? observacoes;
? status;
? created_at;
? updated_at.

atendimento_beneficios

Relaciona atendimentos com beneficios selecionados ou recomendados.

Campos principais:

? atendimento_id;
? beneficio_id;
? created_at.

slots_agendamento

Armazena datas, horarios e capacidade de atendimento.

Campos principais:

? id;
? data;
? horario;
? capacidade;
? vagas_ocupadas;
? ativo.

agendamentos

Armazena agendamentos realizados por usuarios.

Campos principais:

? id;
? atendimento_id;

? slot_id;
? nome;
? telefone;
? status;
? created_at.

7. Rotas da API

Todas as rotas principais estão agrupadas sob o prefixo /api.

7.1 Benefícios

Metodo	Rota	Descrição
GET	/api/beneficios	Lista benefícios ativos.
GET	/api/beneficios/faixas-renda	Lista faixas de renda ativas.

7.2 Agendamentos

Metodo	Rota	Descrição
GET	/api/agendamentos/slots	Lista slots disponíveis, com filtro opcional por data.
GET	/api/agendamentos/datas	Lista datas disponíveis para agendamento.

7.3 Atendimentos

Metodo	Rota	Descrição
POST	/api/atendimentos	Cria um novo atendimento.
GET	/api/atendimentos/:id	Consulta atendimento por ID.
PUT	/api/atendimentos/:id/familia	Atualiza dados familiares.
PUT	/api/atendimentos/:id/beneficios	Salva benefícios selecionados.
GET	/api/atendimentos/:id/beneficios	Consulta benefícios vinculados ao atendimento.
POST	/api/atendimentos/:id/agendar	Cria agendamento para atendimento.

7.4 Health Check

Metodo	Rota	Descrição
GET	/health	Verifica se a API e o banco estão respondendo.

8. Fluxos Principais

8.1 Fluxo de Inicio de Atendimento

```
sequenceDiagram
    participant Usuario
    participant API
    participant Service
    participant Banco

    Usuario->>API: POST /api/atendimentos
    API->>Service: criarAtendimento()
    Service->>Banco: inserir atendimento em_andamento
    Banco-->>Service: atendimento criado
    Service-->>API: dados do atendimento
    API-->>Usuario: 201 Created
```

8.2 Fluxo de Atualizacao Familiar e Elegibilidade

```
sequenceDiagram
    participant Usuario
    participant API
    participant AtendimentoService
    participant ElegibilidadeService
    participant Banco

    Usuario->>API: PUT /api/atendimentos/:id/familia
    API->>AtendimentoService: atualizarDadosFamiliares()
    AtendimentoService->>Banco: validar faixa de renda
    AtendimentoService->>Banco: atualizar atendimento
    AtendimentoService->>ElegibilidadeService: calcular beneficios elegiveis
    ElegibilidadeService->>Banco: consultar beneficios ativos
    Banco-->>ElegibilidadeService: beneficios
    ElegibilidadeService-->>AtendimentoService: beneficios elegiveis
    AtendimentoService-->>API: atendimento + elegibilidade
    API-->>Usuario: 200 OK
```

8.3 Fluxo de Agendamento

```
sequenceDiagram
    participant Usuario
    participant API
    participant Service
    participant Banco

    Usuario->>API: POST /api/atendimentos/:id/agendar
    API->>Service: agendarAtendimento()
    Service->>Banco: buscar slot por data e horario
    Banco-->>Service: slot encontrado
    Service->>Service: validar vagas disponiveis
    Service->>Banco: criar agendamento
    Service->>Banco: incrementar vagas ocupadas
    Service->>Banco: concluir atendimento
    Service-->>API: agendamento criado
    API-->>Usuario: 201 Created
```

8.4 Fluxo de Exibicao de Documentos

```
sequenceDiagram
    participant Usuario
    participant Frontend
    participant API

    Usuario->>Frontend: seleciona beneficios
    Frontend->>API: PUT /api/atendimentos/:id/beneficios
    API-->>Frontend: beneficios selecionados com sucesso
    Usuario->>Frontend: agenda atendimento
    Frontend->>API: POST /api/atendimentos/:id/agendar
```

```
API-->>Frontend: agendamento criado
Frontend->>Frontend: identifica documentos por beneficio
Frontend-->>Usuario: lista documentos necessarios para cadastro
```

Depois do agendamento, a tela exibe a secao **Documentos para cadastro**. Essa secao usa uma tabela local no `app.js`, criada com base no PDF `documentos_programas_sociais.pdf`, para relacionar cada beneficio selecionado aos documentos correspondentes.

9. Como Executar o Projeto

9.1 Pre-requisitos

- ? Node.js instalado;
- ? npm instalado;
- ? banco PostgreSQL/Supabase ativo;
- ? string de conexao do banco;
- ? terminal aberto na pasta `src` do projeto.

9.2 Configurar Variaveis de Ambiente

Na raiz do projeto, crie ou atualize o arquivo `.env`:

```
DATABASE_URL=postgresql://USUARIO:SENHA@HOST.supabase.com:6543/postgres
PORT=3000
```

Observacao: a string real de conexao nao deve ser compartilhada em repositorios publicos.

9.3 Instalar Dependencias

Dentro da pasta `src`, execute:

```
npm install
```

As dependencias principais sao:

```
npm install express pg dotenv
```

9.4 Validar o Banco

```
npm run check:db
```

Esse comando valida se a API consegue se conectar ao PostgreSQL/Supabase.

9.5 Executar Migrations

```
npm run migrate
```

Esse comando cria as tabelas necessarias e insere dados iniciais.

9.6 Validar Codigo

```
npm run build
```

O projeto usa o compilador TypeScript para validar arquivos JavaScript sem gerar saida.

9.7 Iniciar Servidor

```
npm start
```

Por padrao, a API sobe em:

```
http://localhost:3000
```

9.8 Testar pelo Frontend

O servidor Express tambem entrega o frontend estatico localizado em `src/public`.

Depois de iniciar o servidor, abra:

```
http://localhost:3000
```

Fluxo recomendado de teste:

1. Clicar em **Novo atendimento**.
2. Preencher ou manter os dados familiares sugeridos.
3. Clicar em **Salvar e calcular** para gerar beneficios elegiveis.
4. Revisar os beneficios marcados e clicar em **Selecionar marcados**.
5. Escolher data e horario disponivel.
6. Clicar em **Agendar**.
7. Conferir a secao **Documentos para cadastro**, exibida com base nos beneficios selecionados.

10. Validacao e Testes

10.1 Validacoes Realizadas

Foram considerados os seguintes criterios de validacao:

Validacao	Comando	Resultado Esperado
Validacao estatica	<code>npm run build</code>	Sem erros de TypeScript/configuracao
Conexao com banco	<code>npm run check:db</code>	Conexao bem-sucedida ao PostgreSQL
Criacao de tabelas	<code>npm run migrate</code>	Migrations concluidas
Inicializacao da API	<code>npm start</code>	Servidor iniciado sem erro

Validacao	Comando	Resultado Esperado
Frontend estatico	GET http://localhost:3000/	Tela carregada com atendimento, beneficios, agendamento e documentos
JavaScript do frontend	node --check public/app.js	Arquivo sem erros de sintaxe

10.2 Teste Manual do Frontend

O fluxo principal tambem pode ser validado diretamente pela tela:

1. Acessar http://localhost:3000.
2. Criar um novo atendimento.
3. Salvar dados familiares e calcular beneficios elegiveis.
4. Selecionar os beneficios desejados.
5. Realizar o agendamento.
6. Verificar se a secao **Documentos para cadastro** aparece com os documentos correspondentes aos beneficios selecionados.

10.3 Testes Manuais de API

Exemplos de chamadas que podem ser feitas com Postman, Insomnia ou Thunder Client.

Criar atendimento

```
POST http://localhost:3000/api/atendimentos
Content-Type: application/json
```

Resposta esperada:

```
{
  "sucesso": true,
  "data": {
    "id": 1,
    "status": "em_andamento"
  }
}
```

Listar beneficios

```
GET http://localhost:3000/api/beneficios
```

Resposta esperada:

```
{
  "sucesso": true,
  "data": []
}
```

Atualizar dados familiares

```
PUT http://localhost:3000/api/atendimentos/1/familia
Content-Type: application/json
```

```
{
  "faixaRendaId": 1,
  "quantidadePessoas": 4,
}
```

```

    "quantidadeAdultos": 2,
    "quantidadeCrianças": 2,
    "quantidadeIdosos": 0,
    "possuiPcd": false,
    "observacoes": "Familia busca orientacao sobre beneficios."
  }

```

Resposta esperada:

```

{
  "sucesso": true,
  "data": {
    "atendimento": {},
    "beneficiosElegiveis": []
  }
}

```

Criar agendamento

POST http://localhost:3000/api/atendimentos/1/agendar
Content-Type: application/json

```

{
  "nome": "Maria Silva",
  "telefone": "11999999999",
  "data": "2026-05-25",
  "horario": "09:00"
}

```

Resposta esperada:

```

{
  "sucesso": true,
  "data": {
    "id": 1,
    "status": "agendado"
  }
}

```

10.4 Suite de Testes Recomendada

Para evoluir o projeto, recomenda-se criar testes Jest de integracao usando banco real de teste.

Casos minimos recomendados:

Caso	Objetivo
Sucesso	Criar atendimento, atualizar familia e receber beneficios elegiveis.
Regra de negocio violada	Tentar agendar horario sem vagas disponiveis.
Payload invalido	Enviar dados familiares incompletos ou beneficios em formato invalido.
Persistencia no banco	Verificar se agendamento criado foi salvo e se vagas ocupadas aumentaram.

11. Matriz RF x RN x Endpoint

RF	Requisito	RN Relacionada	Endpoint
RF01	Iniciar atendimento	RN01	POST /api/atendimentos
RF02	Consultar beneficios	RN04	GET /api/beneficios

RF	Requisito	RN Relacionada	Endpoint
RF03	Consultar faixas de renda	RN03	GET /api/beneficios/faixas-renda
RF04	Atualizar dados familiares	RN02, RN03	PUT /api/atendimentos/:id/familia
RF05	Calcular beneficios elegiveis	RN05	PUT /api/atendimentos/:id/familia
RF06	Selecionar beneficios	RN06	PUT /api/atendimentos/:id/beneficios
RF07	Consultar beneficios de atendimento	RN04, RN06	GET /api/atendimentos/:id/beneficios
RF08	Consultar datas disponiveis	RN08	GET /api/agendamentos/datas
RF09	Consultar slots disponiveis	RN08	GET /api/agendamentos/slots
RF10	Criar agendamento	RN07, RN08, RN09, RN10	POST /api/atendimentos/:id/agendar
RF11	Consultar atendimento	RN01	GET /api/atendimentos/:id
RF12	Verificar saude da API	RN12	GET /health
RF13	Exibir documentos necessarios	RN13	Frontend src/public/app.js

12. Matriz de Riscos

Risco	Probabilidade	Impacto	Mitigacao
Instabilidade na conexao com o banco	Media	Alto	Health check, tratamento de erro e validacao de DATABASE_URL.
Exposicao da string de conexao	Baixa	Alto	Uso de .env e .env.example sem segredo real.
Agendamentos acima da capacidade	Media	Alto	Validacao de slot e incremento de vagas ocupadas no banco.
Critérios de beneficios incompletos	Media	Medio	Manter criterios em estrutura configuravel e revisar com especialistas.
Dados familiares inconsistentes	Media	Medio	Validacao de payload antes de persistir.
Baixa acessibilidade no front do totem	Media	Alto	Projetar interface com linguagem simples, contraste e fluxo guiado.
Falta de integracao com sistemas oficiais	Alta	Medio	Planejar integracoes futuras via API ou importacao controlada.

13. Conclusao e Trabalhos Futuros

O projeto entrega a base backend e uma interface simples para um fluxo de autoatendimento socioassistencial. A aplicacao ja possui estrutura em camadas, persistencia em banco real, migrations, endpoints principais, frontend estatico e regras de negocio relacionadas a atendimento, elegibilidade, agendamento e exibicao de documentos.

A solucao contribui para reduzir barreiras de acesso, organizar a triagem inicial e preparar melhor o atendimento realizado pela equipe publica.

Como trabalhos futuros, recomenda-se:

- ? criar suite Jest de integracao com banco real de teste;
- ? adicionar autenticao para funcionarios e administradores;
- ? criar painel administrativo para gestao de slots e beneficios;
- ? integrar notificacoes por WhatsApp, SMS ou e-mail;
- ? registrar logs estruturados de atendimento;
- ? implementar auditoria de alteracoes;
- ? conectar o sistema a bases oficiais da prefeitura;
- ? evoluir regras de elegibilidade com apoio de especialistas da area social;
- ? evoluir o frontend para uma experiencia final de totem com acessibilidade, leitura facilitada e suporte a impressao controlada.

14. Referencias

- ? Documentacao oficial do Node.js;
- ? Documentacao oficial do Express;
- ? Documentacao oficial do PostgreSQL;
- ? Documentacao oficial do Supabase;
- ? Repositorio do projeto HackthonGuarulhos;
- ? Arquivos internos do projeto: controllers, services, repositories, migrations, rotas e frontend em src/public;
- ? PDF documentos_programas_sociais.pdf, usado como base para a lista de documentos por programa social;
- ? Documento WAD de referencia fornecido como base de formatacao.